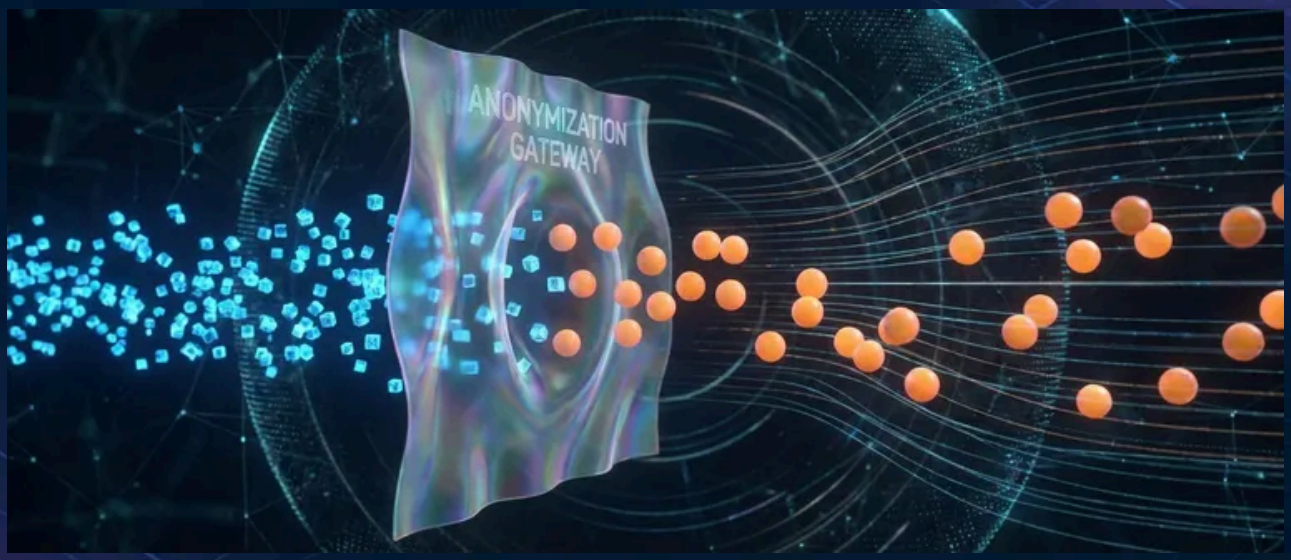
Test Data Without PII Leaks



Published on December 14, 2024

Webstack Builders

Table of Contents

Three weeks into a new engagement, I found 47,000 real customer records in the staging database. Names, emails, phone numbers, addresses — everything. The dev team's rationale was familiar: "We needed realistic data for testing." When I asked about their data retention policy for non-production environments, the room went quiet. That staging database had been copied from production eighteen months ago and never refreshed. It had survived three contractor rotations, two security group misconfigurations, and one laptop theft. Nobody knew where copies of it lived.

This is the production data trap. Copying real data feels like the path of least resistance, but it creates legal liability that compounds silently until an auditor or attacker finds it.

The Production Data Trap

I understand the justification for using production data in testing. "We need realistic data." "Our schema is too complex to fake." "We need to reproduce production bugs." "Performance testing requires real volumes." Each sounds reasonable until you examine it.

Realistic data doesn't require *real* data. Synthetic generators with proper domain logic produce data that's indistinguishable from production for testing purposes. Complex schemas are exactly where you *need* synthetic generators — if you can't generate valid test data, you've built something you can't actually test. Reproducing bugs requires the *pattern* that triggered the bug, not the actual customer's information. Performance testing cares about volume and distribution, both of which synthetic data handles fine.

The most dangerous justification is "it's just staging — nobody will see it." Staging environments get breached. Security groups get misconfigured. Laptops get stolen. Contractors rotate in and out. Compliance doesn't care about intent; it cares about where customer data ended up.

The Cost of PII in Test Data

Every non-production environment with real data is a breach waiting to happen. The exposure surface is larger than most teams realize.

#	Risk	Consequence	Real-World Example
1	Data breach in staging	GDPR fines up to €20M or 4% revenue	Staging DB exposed via misconfigured security group

#	Risk	Consequence	Real-World Example
2	Developer laptop theft	Customer data compromise, notification requirements	Laptop with local DB copy stolen from car
3	Log aggregation exposure	PII in logs indexed by third-party service	Customer names in error messages sent to Datadog
4	Contractor access	PII exposure to untrusted parties	Offshore QA team with full production clone
5	Backup retention	Production data in test backups subject to retention	Test DB backup stored for years, breached
6	Compliance audit failure	Failed SOC2/HIPAA audit, lost enterprise deals	Auditor finds real SSNs in staging environment

Common PII exposure scenarios in non-production

Most teams have no inventory of where production copies live. A single database export fans out to developer laptops, CI artifacts, log aggregators, error trackers, and backup systems — each one a potential breach vector.

DANGER

Under GDPR (General Data Protection Regulation), using production personal data for testing without explicit consent violates the purpose limitation principle. HIPAA (Health Insurance Portability and Accountability Act) requires PHI (Protected Health Information) to be de-identified before use in development. PCI (Payment Card Industry) DSS (Data Security Standard) prohibits real card numbers in non-production. The convenience isn't worth the risk.

Synthetic Data Generation

If production data is off-limits, what's the alternative? Synthetic data — generated from scratch with no connection to real customers. But "just use Faker" understates the challenge.

Generating individual fake records is the easy part. The hard part is understanding the relationships between tables well enough to generate data that won't violate constraints or produce nonsensical combinations.

The Real Challenge: Schema Relationships

Most tutorials show you how to generate a fake user with Faker. That's trivial. The actual problem is generating an order that references a valid user, with line items that reference valid products, shipping to an address that belongs to that user, with a payment method that's active and owned by that same user, charged to a billing address that may or may not match the shipping address. Miss any of these relationships and your tests either fail on constraint violations or pass with data that could never exist in production.

Legacy databases make this worse. They accumulate implicit relationships that aren't enforced by foreign keys — columns named `customer_id` that reference the `users` table instead of `customers`, lookup tables with undocumented code values, soft deletes that leave orphaned references. Before you can generate valid synthetic data, you need to understand what valid data actually looks like.

Discovering Schema Relationships from Production

When documentation is missing or wrong, the database itself is the source of truth. Here's how to extract the actual relationship graph from a PostgreSQL database:

schema_discovery.py

```
# Extract foreign key relationships from PostgreSQL information_schema
import psycopg2

def get_foreign_keys(conn) -> list[dict]:
    query = """
    SELECT
        tc.table_name AS source_table,
        kcu.column_name AS source_column,
        ccu.table_name AS target_table,
        ccu.column_name AS target_column
    FROM information_schema.table_constraints AS tc
    JOIN information_schema.key_column_usage AS kcu
        ON tc.constraint_name = kcu.constraint_name
    JOIN information_schema.constraint_column_usage AS ccu
        ON ccu.constraint_name = tc.constraint_name
```

```
        WHERE tc.constraint_type = 'FOREIGN KEY'
        """
        with conn.cursor() as cur:
            cur.execute(query)
            return [dict(zip(['source_table', 'source_column',
                              'target_table', 'target_column'], row))
                    for row in cur.fetchall()]
```

Extracting declared foreign keys from PostgreSQL

Foreign keys only tell part of the story. Many relationships exist by convention rather than constraint. To find these implicit references, look for columns that share names with primary keys in other tables:

schema_discovery.py

```
# Find likely implicit foreign keys by naming convention
def find_implicit_relationships(conn, explicit_fks: list[dict]) -> list[dict]:
    # Get all columns ending in _id
    query = """
    SELECT table_name, column_name
    FROM information_schema.columns
    WHERE column_name LIKE '%_id'
      AND table_schema = 'public'
    """
    with conn.cursor() as cur:
        cur.execute(query)
        candidate_columns = cur.fetchall()

    # Get all primary keys
    pk_query = """
    SELECT tc.table_name, kcu.column_name
    FROM information_schema.table_constraints tc
    JOIN information_schema.key_column_usage kcu
        ON tc.constraint_name = kcu.constraint_name
    WHERE tc.constraint_type = 'PRIMARY KEY'
    """
    with conn.cursor() as cur:
        cur.execute(pk_query)
        primary_keys = {row[0]: row[1] for row in cur.fetchall()}

```

```
    implicit = []
    for table, column in candidate_columns:
        # user_id likely references users.id
        likely_table = column.replace('_id', 's')
        if likely_table in primary_keys:
            # Check it's not already an explicit FK
            if not any(fk['source_table'] == table and
                       fk['source_column'] == column for fk in explicit_fks):
                implicit.append({
                    'source_table': table,
                    'source_column': column,
                    'target_table': likely_table,
                    'confidence': 'naming_convention'
                })
    return implicit
```

Discovering implicit relationships by naming conventions

These heuristics catch common patterns, but they miss relationships where naming conventions weren't followed. The next step is to validate candidates by checking if values actually reference existing records:

schema_discovery.py

```
# Validate implicit relationships by checking referential integrity
def validate_relationship(conn, source_table: str, source_col: str,
                          target_table: str, target_col: str) -> float:
    query = f"""
    SELECT
        COUNT(*) AS total,
        COUNT(CASE WHEN t.{target_col} IS NOT NULL THEN 1 END) AS matched
    FROM {source_table} s
    LEFT JOIN {target_table} t ON s.{source_col} = t.{target_col}
    WHERE s.{source_col} IS NOT NULL
    """
    with conn.cursor() as cur:
        cur.execute(query)
        total, matched = cur.fetchone()
    return matched / total if total > 0 else 0
```

Measuring referential integrity to validate candidate relationships

A match rate above 95% strongly suggests a real relationship. Lower rates might indicate soft deletes, historical data, or false positives in your naming heuristic.

Generating Data in Dependency Order

Once you understand the schema, you need to generate tables in the right order. You can't create an order before the user it references exists. This requires topological sorting of the dependency graph:

fixture_generator.py

```python
from collections import defaultdict
from faker import Faker

fake = Faker()

def topological_sort(relationships: list[dict]) -> list[str]:
    """Return tables sorted so dependencies come before dependents."""
    graph = defaultdict(set)
    all_tables = set()

    for rel in relationships:
        graph[rel['source_table']].add(rel['target_table'])
        all_tables.add(rel['source_table'])
        all_tables.add(rel['target_table'])

    visited = set()
    order = []

    def visit(table):
        if table in visited:
            return
        visited.add(table)
        for dep in graph[table]:
            visit(dep)
        order.append(table)

    for table in all_tables:
        visit(table)

    return order  # Dependencies first, then dependents
```

Topological sort to determine table generation order

With the generation order established, you can build fixtures that respect referential integrity. The key insight is maintaining a registry of generated primary keys so child tables can reference valid parents:

fixture_generator.py

```python
class FixtureRegistry:
    """Track generated records for referential integrity."""

    def __init__(self):
        self.records: dict[str, list[dict]] = defaultdict(list)

    def add(self, table: str, record: dict):
        self.records[table].append(record)

    def get_random_id(self, table: str, id_column: str = 'id'):
        """Get a random existing ID for foreign key references."""
        if not self.records[table]:
            raise ValueError(f"No records in {table} to reference")
        record = fake.random_element(self.records[table])
        return record[id_column]
```

Registry pattern for tracking generated records

Basic Faker Generation

With the infrastructure in place, individual record generation becomes straightforward. Here's a simple user generator:

generators/user.py

```python
from faker import Faker

fake = Faker()
Faker.seed(12345)  # Reproducible fixtures

def generate_user() -> dict:
    first = fake.first_name()
    last = fake.last_name()
    return {
        'id': fake.uuid4(),
```

```
        'email': f"{first.lower()}.{last.lower()}@example.com",
        'first_name': first,
        'last_name': last,
        'created_at': fake.date_time_between(start_date='-2y'),
    }
```

Basic user generation with Faker

The `Faker.seed()` call ensures you get the same records every time, which matters for snapshot testing and debugging. Without deterministic generation, a test failure might not reproduce because the next run generates different data.

Production Data Anonymization

Sometimes you genuinely need production data — debugging a specific customer issue, reproducing a complex data pattern, or performance testing with realistic distributions. The answer isn't "never use production data." It's "never use *identifiable* production data." Anonymization lets you keep the structure and relationships while removing the liability.

Choosing the Right Technique

Different data types need different anonymization approaches. The goal is preserving what matters for testing while destroying what identifies individuals.

#	Field Type	Technique	Example	What's Preserved
1	Email	Format-preserving hash	john.doe@company.com → user_8f3k2@example.com	Format validity, uniqueness
2	Name	Consistent fake replacement	John Doe → Michael Smith	Linguistic plausibility
3	Phone	Format-preserving randomization	555-123-4567 → 555-987-6543	Format, area code patterns

#	Field Type	Technique	Example	What's Preserved
4	Address	Generalization + fake details	123 Main St, Springfield IL → 456 Oak Ave, Springfield IL	Geographic region
5	Date of Birth	Bucketing	1985-03-15 → 1985-01-01	Age range, decade
6	SSN/ID numbers	Complete replacement	123-45-6789 → 987-65-4321	Format validity only
7	Free text	NLP entity replacement	"John called about his order" → "Customer called about their order"	Sentence structure

Anonymization techniques by field type

The critical insight is *consistency*: the same real value must always map to the same fake value. Without this, referential integrity breaks. If `john@real.com` appears in both the `users` and `orders` tables, both occurrences need to become the same anonymized value.

Deterministic Anonymization

Consistency requires deterministic transformation. Hash the original value to generate a seed, then use that seed to produce the fake value. Same input, same output, every time:

anonymizer.py

```
import hashlib
from faker import Faker

class ConsistentAnonymizer:
    def __init__(self, salt: str):
        self.salt = salt
        self.fake = Faker()

    def _get_seed(self, field: str, value: str) -> int:
        """Generate deterministic seed from field name and value."""
```

```
            combined = f"{self.salt}:{field}:{value}"
            hash_hex = hashlib.sha256(combined.encode()).hexdigest()
            return int(hash_hex[:8], 16)

        def anonymize_email(self, email: str) -> str:
            seed = self._get_seed('email', email)
            Faker.seed(seed)
            username = self.fake.user_name().lower()
            return f"{username}@example.com"
```

Deterministic anonymization using seeded Faker

The salt is important — it prevents someone with access to both production and anonymized data from reverse-engineering the mapping. Rotate salts between environments.

With this foundation, you can anonymize related tables while preserving joins:

anonymizer.py

```
def anonymize_records(self, records: list[dict],
                      field_mappings: dict[str, str]) -> list[dict]:
    """Anonymize specified fields across all records consistently."""
    result = []
    for record in records:
        anonymized = record.copy()
        for field, technique in field_mappings.items():
            if field in record and record[field]:
                anonymized[field] = getattr(self, f'anonymize_{technique}')(
                    record[field]
                )
        result.append(anonymized)
    return result

# Same email in users and orders maps to same fake email
anonymizer = ConsistentAnonymizer(salt='prod-to-staging-2024')
users = anonymizer.anonymize_records(raw_users, {'email': 'email', 'name': 'name'})
orders = anonymizer.anonymize_records(raw_orders, {'customer_email': 'email'})
```

Preserving referential integrity across tables

Database-Level Anonymization

For large datasets, anonymizing in Python row-by-row is slow. PostgreSQL functions let you transform data at the database level during export:

anonymization_functions.sql

```sql
-- PostgreSQL: Deterministic email anonymization
CREATE OR REPLACE FUNCTION anon.fake_email(real_email TEXT)
RETURNS TEXT AS $$
DECLARE
  hash_val TEXT;
BEGIN
  hash_val := md5(real_email || current_setting('anon.salt'));
  RETURN 'user_' || substring(hash_val from 1 for 8) || '@example.com';
END;
$$ LANGUAGE plpgsql IMMUTABLE;
```

PostgreSQL anonymization function

The `IMMUTABLE` marker is essential — it tells PostgreSQL the function always returns the same output for the same input, enabling query optimization. Set the salt via `SET anon.salt = 'your-secret'` before running exports.

With functions in place, create an anonymized view for export:

anonymized_export.sql

```sql
-- Anonymized view for staging data export
CREATE VIEW staging.users_export AS
SELECT
  id,
  anon.fake_email(email) AS email,
  anon.fake_name(full_name) AS full_name,
  status,
  created_at,
  date_trunc('month', date_of_birth) AS date_of_birth,
  NULL AS ssn  -- Completely remove, don't anonymize
```

```
  FROM production.users;

  COPY (SELECT * FROM staging.users_export)
  TO '/tmp/users_anonymized.csv' WITH CSV HEADER;
```

Creating anonymized export views in PostgreSQL

Some fields shouldn't be anonymized — they should be nullified. SSNs, credit card numbers, and security questions have no testing value that justifies keeping even a transformed version in most cases.

> **INFO**
>
> The exception is when you're testing validation logic, display masking, or payment processor integrations. For these cases, don't derive fake SSNs or card numbers from production values — generate them fresh with no relationship to real data. Faker's `ssn()` and `credit_card_number()` methods produce valid-format values that pass regex and checksum validation. The key distinction: anonymization transforms real data into fake data (creating a mapping that could theoretically be reversed), while generation creates fake data from nothing. For highly sensitive fields, generation is safer than anonymization.

The Re-identification Risk

Anonymization isn't a magic wand. Even with names and emails replaced, a record with unique purchase history, specific timestamps, and geographic patterns can still identify someone. This is called a **quasi-identifier attack**.

> **WARNING**
>
> Consider k-anonymity: each record should be indistinguishable from at least `k-1` other records on quasi-identifier fields (zip code, birth year, gender combinations). A common target is `k=5`, meaning every combination of quasi-identifiers appears at least 5 times.[1] If your anonymized dataset contains a record for the only 1987-born customer in zip code 90210 who ordered on December 25th, that's effectively identified regardless of the fake name attached.

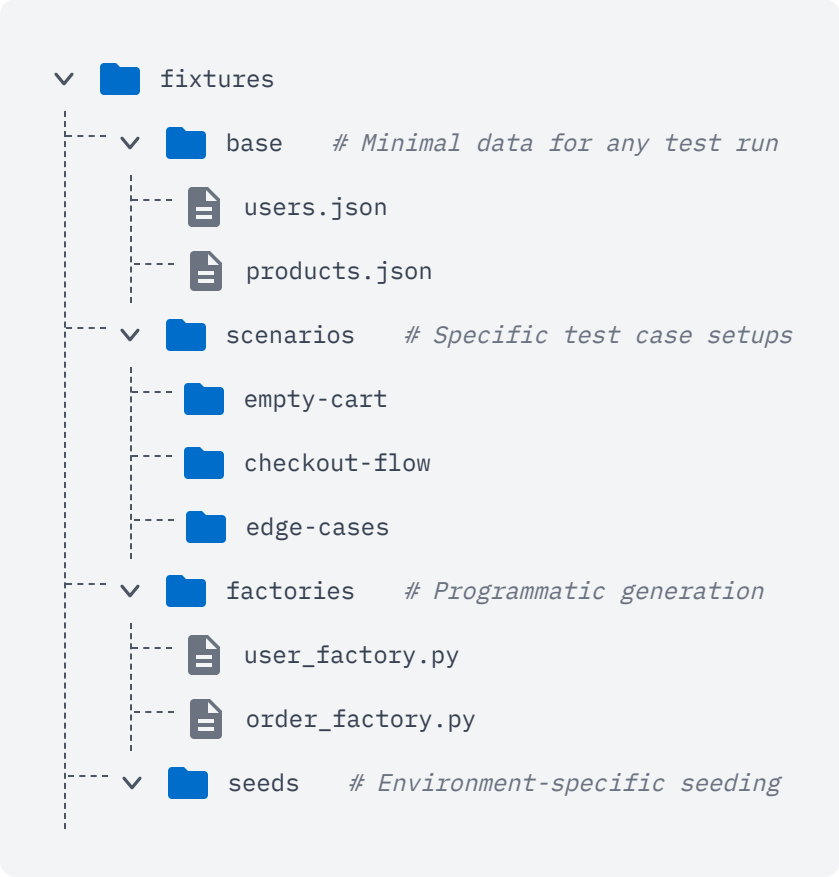
For high-sensitivity data, consider generalization beyond simple field replacement: bucket timestamps to week granularity, truncate zip codes to the first 3 digits (which covers a broader geographic area), remove or add noise to low-cardinality combinations. The tradeoff is reduced testing fidelity — a 3-digit zip won't validate against a 5-digit regex, and weekly timestamps break tests that depend on exact ordering. But that's often acceptable compared to the re-identification risk.

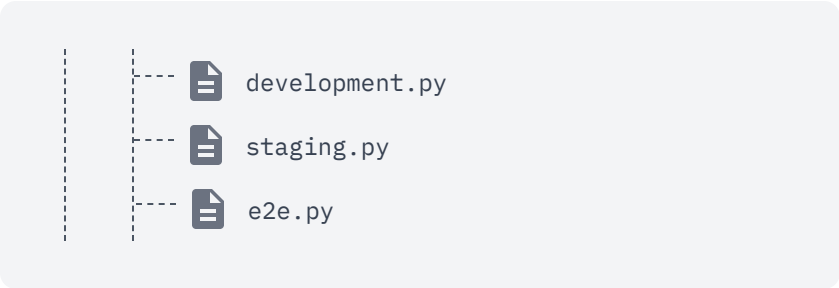
Fixture Management

Test data isn't a one-time problem. Schemas evolve, test scenarios multiply, and fixtures drift out of sync with production reality. Without deliberate organization, you end up with test data scattered across the codebase — some in JSON files, some hardcoded in tests, some generated on the fly with different seeds.

Organizing Fixtures by Purpose

The most maintainable approach separates fixtures by their role in testing. Base fixtures provide the minimal data every test needs. Scenario fixtures set up specific test cases. Factories generate data programmatically when you need flexibility or volume.





Fixture directory structure separating concerns.

Base fixtures are checked into version control and rarely change. They represent the "happy path" data that most tests assume exists — a default user, a few products, standard configuration. Keep them minimal; tests that need specific setups should use scenario fixtures or factories.

The Factory Pattern

Static JSON fixtures work for simple cases, but they become unwieldy when tests need variations. A factory encapsulates the logic for generating valid records with sensible defaults, while allowing tests to override specific fields:

factories/user_factory.py

```
from faker import Faker
from dataclasses import dataclass, field
from datetime import datetime

fake = Faker()

@dataclass
class UserFactory:
    id: str = field(default_factory=lambda: fake.uuid4())
    email: str = field(default_factory=lambda: fake.email())
    first_name: str = field(default_factory=lambda: fake.first_name())
    status: str = 'active'
    tier: str = 'free'
    created_at: datetime = field(default_factory=datetime.now)

    def build(self, **overrides) -> dict:
        """Generate a user dict with optional field overrides."""
        data = {k: v for k, v in self.__dict__.items()}
```

```
        data.update(overrides)
        return data
```

Factory pattern for flexible test data generation

Tests then read clearly — you see exactly what's different about this test's data:

tests/test_dashboard.py

```
def test_shows_upgrade_prompt_for_free_users(db):
    user = UserFactory().build(tier='free')
    db.insert('users', user)
    # ... test assertions

def test_hides_upgrade_prompt_for_enterprise_users(db):
    user = UserFactory().build(tier='enterprise')
    db.insert('users', user)
    # ... test assertions
```

Tests using factories communicate intent clearly

Keeping Fixtures in Sync with Schema

The most insidious fixture bug is when your schema evolves but your fixtures don't. Tests pass because they're testing against stale data structures that no longer match production. You find out when the code hits real data.

One approach is generating fixtures from the schema itself. If you're using an ORM like SQLAlchemy, you can introspect the models to validate fixtures at test startup:

fixtures/validator.py

```
from sqlalchemy import inspect

def validate_fixture(model_class, fixture_data: dict) -> list[str]:
    """Check fixture against SQLAlchemy model columns."""
    mapper = inspect(model_class)
```

```
    model_columns = {col.key for col in mapper.columns}
    fixture_keys = set(fixture_data.keys())

    errors = []
    missing = model_columns - fixture_keys - {'id', 'created_at', 'updated_at'}
    extra = fixture_keys - model_columns

    if missing:
        errors.append(f"Fixture missing required columns: {missing}")
    if extra:
        errors.append(f"Fixture has unknown columns: {extra}")
    return errors
```

Validating fixtures against ORM schema

Run this validation in CI. When someone adds a required column, tests fail immediately rather than silently using incomplete data.

Compliance Considerations

Synthetic data isn't just a technical convenience — it's often a legal requirement. If you're handling customer data, you're almost certainly subject to regulations that restrict how that data can be used outside production.

Regulatory Landscape

Different regulations have different specifics, but the common thread is clear: personal data in test environments is either prohibited outright or requires controls most teams don't have.

Regulation	Test Data Requirement	Key Provision
GDPR	Cannot use personal data without consent	Article 5(1)(b) - Purpose limitation
CCPA	Must honor opt-out for non-essential processing	§1798.120 - Right to opt out
HIPAA	PHI must be de-identified (Safe Harbor or Expert)	§164.514 - De-identification standard
PCI DSS	No real PANs in non-production	Requirement 6.4.3
SOX	Data accuracy for financial testing	§302 - Corporate responsibility

Regulation	Test Data Requirement	Key Provision
FERPA	Student records must be de-identified	§99.31 - Disclosure requirements

Regulatory requirements for test data

The GDPR (General Data Protection Regulation)'s "purpose limitation" principle is particularly strict: data collected for providing a service cannot be repurposed for testing without explicit consent. Arguing that testing improves the service won't satisfy regulators. HIPAA (Health Insurance Portability and Accountability Act)'s de-identification standard requires either removing 18 specific identifier types (Safe Harbor <https://www.hhs.gov/hipaa/for-professionals/special-topics/de-identification/index.html#safeharborguidance>) or having a statistician certify re-identification risk is "very small" (Expert Determination <https://www.hhs.gov/hipaa/for-professionals/special-topics/de-identification/index.html#guidancedetermination>). PCI (Payment Card Industry) DSS (Data Security Standard) is blunt — real card numbers in non-production environments is a compliance failure, full stop.

Automated PII Scanning

Trust but verify. Even with synthetic data generation in place, run automated scans before seeding any environment. Patterns slip through — a developer hardcodes a test email with a real domain, a fixture file gets accidentally committed with production data, an anonymization function has a bug.

compliance_scanner.py

```python
import re
import json

def scan_for_pii(data: dict | list) -> list[dict]:
    """Scan serialized data for common PII patterns."""
    findings = []
    text = json.dumps(data)

    # Real email addresses (not @example.com or @test.com)
    emails = re.findall(
        r'[a-z0-9._%+-]+@(?!example\.com|test\.com)[a-z0-9.-]+\.[a-z]{2,}',
        text, re.IGNORECASE
    )
    if emails:
        findings.append({'type': 'email', 'count': len(emails),
```

```
                        'samples': emails[:3]})

    # SSN patterns
    ssns = re.findall(r'\b\d{3}-\d{2}-\d{4}\b', text)
    if ssns:
        findings.append({'type': 'ssn', 'count': len(ssns),
                         'samples': ['***-**-****'] * min(3, len(ssns))})

    return findings
```

Basic PII (Personally Identifiable Information) pattern scanner

This catches obvious violations. For credit card numbers, you'll want Luhn validation to distinguish real cards from random digit sequences:

compliance_scanner.py

```
def is_luhn_valid(card_number: str) -> bool:
    """Check if card number passes Luhn checksum."""
    digits = [int(d) for d in card_number if d.isdigit()]
    checksum = 0
    for i, digit in enumerate(reversed(digits)):
        if i % 2 == 1:
            digit *= 2
            if digit > 9:
                digit -= 9
        checksum += digit
    return checksum % 10 == 0

def scan_for_cards(data: dict | list) -> list[str]:
    """Find Luhn-valid card numbers in data."""
    text = json.dumps(data)
    # Visa, Mastercard, Amex patterns
    candidates = re.findall(
        r'\b(?:4[0-9]{15}|5[1-5][0-9]{14}|3[47][0-9]{13})\b', text
    )
    return [c for c in candidates if is_luhn_valid(c)]
```

Credit card detection with Luhn validation

Integrate scanning into your CI pipeline. If the scan finds anything, fail the build:

ci_compliance_check.py

```python
import sys

def run_compliance_check(fixture_path: str) -> int:
    with open(fixture_path) as f:
        data = json.load(f)

    pii = scan_for_pii(data)
    cards = scan_for_cards(data)

    if pii or cards:
        print("COMPLIANCE FAILURE - PII detected in fixtures")
        for finding in pii:
            print(f"  {finding['type']}: {finding['count']} instances")
        if cards:
            print(f"  credit_cards: {len(cards)} Luhn-valid numbers")
        return 1
    return 0

if __name__ == '__main__':
    sys.exit(run_compliance_check(sys.argv[1]))
```

CI integration for compliance scanning

Audit Trails

When an auditor asks “how do you ensure no customer data in testing?”, you need more than a verbal explanation. You need evidence: policy documents, technical controls, and logs proving those controls are followed.

Log every test data operation with enough context to reconstruct what happened:

audit_logger.py

```python
import json
import os
from datetime import datetime

def log_seed_operation(environment: str, source: str,
```

```
                        record_counts: dict, scan_passed: bool):
    entry = {
        'timestamp': datetime.utcnow().isoformat(),
        'environment': environment,
        'action': 'seeded',
        'data_source': source,  # 'synthetic' or 'anonymized_production'
        'record_counts': record_counts,
        'operator': os.environ.get('CI_JOB_NAME', os.getlogin()),
        'compliance_scan': 'passed' if scan_passed else 'failed',
    }
    with open('test-data-audit.jsonl', 'a') as f:
        f.write(json.dumps(entry) + '\n')
```

Audit logging for test data operations

Keep these logs alongside your application audit logs. During a compliance review, being able to show a timestamped record of every staging database refresh — with compliance scan results — demonstrates mature data handling practices.

Auditors look for evidence of process, not just policy. A documented policy saying “we use synthetic data” isn’t sufficient. You need:

- The policy document
- Technical controls enforcing it (scanners)
- Logs proving controls ran
- Evidence of periodic review

Synthetic data with this documentation passes audits easily.

Conclusion

Production data in non-production environments is a liability disguised as convenience. Understand your schema deeply enough to generate valid synthetic data — both explicit foreign keys and implicit relationships. When you genuinely need production patterns, anonymize through automated pipelines with deterministic

transformations, but remember that highly sensitive fields should be generated fresh rather than derived from real values.

Organize fixtures by purpose, validate them against your schema in CI, and run compliance scans before every seed operation. Build generators as you build features — don't wait for a compliance incident to retrofit synthetic data onto a codebase addicted to production copies.

---

1. The choice of `k` depends on your risk tolerance and data sensitivity. Healthcare data often requires k≥10 or higher. For general business data, `k=5` is a reasonable starting point. Higher `k` values require more aggressive generalization, which reduces data utility for testing. ↩

https://www.webstackbuilders.com/articles/synthetic-test-data-pii-anonymization-fixtures